Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Integrantes	María Valarezo Cristopher Chingo Santiago Villamagua Mauricio Guachizaca Mateo Silva
Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C.
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe
Modalidad de trabajo	Cada estudiante simula en su computadora; se trabaja y se entrega en parejas; la prueba física se hace con la placa del grupo
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica:

Objetivo. Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Fundamento teórico

El ESP32 DevKit V1 de 30 pines usa el módulo ESP32-WROOM-32 de Espressif: un microcontrolador que trabaja con lógica de 3,3 V y cuya CPU funciona hasta a 240 MHz. Se programa desde Arduino IDE con el paquete esp32 by Espressif Systems, con las funciones habituales de Arduino: pinMode(), digitalWrite(), digitalRead() y millis().

Temporización no bloqueante. delay(ms) detiene la ejecución de loop() durante el tiempo indicado. millis() devuelve los milisegundos transcurridos desde el arranque; si se compara ahora - tPaso >= PERIODO, la acción se ejecuta cada cierto tiempo sin detener el programa, y loop() puede seguir atendiendo otras entradas mientras tanto.

Lectura del pulsador. Con pinMode(pin, INPUT_PULLUP) se activa la resistencia pull-up interna del pin, por lo que el pulsador puede conectarse directamente entre el GPIO y GND, sin resistencia externa. Los contactos mecánicos de un pulsador rebotan durante unos milisegundos al cambiar de estado; el antirrebote por software acepta un cambio solo cuando la señal se mantiene estable durante un tiempo (T_REBOTE), y la detección de flanco actúa una sola vez, en el instante en que el estado filtrado pasa de suelto a presionado.

Elección de pines. En el ESP32 conviene evitar los pines de strapping (0, 2, 5, 12, 15), los pines solo de entrada (34–39) y los que usa el USB (1 y 3). En la placa de 30 pines, los GPIO 16 y 17 aparecen serigrafiados como RX2 y TX2, no con su número.

4. Materiales y reactivos:

Hardware (por grupo, solo para la Parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard
4	LEDs de 5 mm rojos, amarillos o verdes (no azules ni blancos: casi no encienden a 3,3 V)
4	Resistencias de 330 Ω
1	Pulsador de 4 patas
6	Cables jumper macho-macho

5. Equipos y herramientas

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE 2.x con el paquete esp32 by Espressif Systems versión 3.x, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

6. Procedimiento / Metodología

Práctica de 3 h en parejas: cuatro LEDs avanzan en secuencia sin usar delay() y un pulsador cambia la velocidad. Se hace en Wokwi y, si el grupo ya tiene su placa, también en el ESP32 real. Sin reto de extensión.

Tabla de conexiones

Todos los pines elegidos son seguros: no son de strapping (0, 2, 5, 12, 15), ni solo de entrada (34–39), ni los usa el USB (1 y 3).

Componente	GPIO	Serigrafía en la placa	Conexión
LED 1	16	RX2	GPIO → resistencia 330 Ω → ánodo (pata larga); cátodo (pata corta) → GND
LED 2	17	TX2	igual que el LED 1
LED 3	18	D18	igual que el LED 1

LED 4	19	D19	igual que el LED 1
Pulsador	21	D21	una pata al GPIO21 y la pata del lado opuesto a GND; sin resistencia externa (usa la pull-up interna)
Tierra	—	GND	línea azul (−) de la protoboard; ahí van los cátodos y el pulsador

En la placa de 30 pines, los GPIO 16 y 17 aparecen como RX2 y TX2, no con su número. Comparen siempre con la serigrafía de su propia placa: los clones pueden cambiar el orden.

Parte A. Circuito y secuencia de luces (Wokwi)

En esta parte los cuatro LEDs se encienden uno tras otro, sin usar delay().

A1. Entren a wokwi.com y creen un proyecto nuevo de ESP32.

A2. Abran la pestaña diagram.json, borren todo su contenido y peguen el Anexo A. Aparecen la placa, los cuatro LEDs, las resistencias y el pulsador ya cableados.

A3. Revisen que cada cable coincida con la Tabla de conexiones. Ubiquen en la placa los pines RX2, TX2, D18, D19 y D21.

A4. Abran la pestaña sketch.ino, borren su contenido, peguen el Código 1 y presionen ▸ (Start).

Resultado esperado: un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16... cada 0,3 s.

A5. Cambien PERIODO a 1000 y luego a 100. Ejecuten cada vez y completen la Tabla A.

A6. Vuelvan a dejar PERIODO = 300 y tomen una captura de pantalla del circuito funcionando.

Código 1. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías

```
// AP2 - Codigo 1: secuencia de 4 LEDs sin delay()
// Placa: ESP32 Dev Module | core Espressif 3.x | sin librerias
// Pines: LEDs en GPIO16, 17, 18 y 19 (cada uno con 330 ohm)
constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
constexpr uint32_t PERIODO = 300;    // ms entre un paso y el siguiente

uint8_t posicion = 0;
                                     // LED encendido (0 a 3)
uint32_t tPaso = 0;                  // instante del ultimo
                                     paso

void setup() {
  for (uint8_t i = 0; i < N_LEDS; i++) {
    pinMode(LEDS[i], OUTPUT);
  }
}

void loop() {
  uint32_t ahora = millis();

  if (ahora - tPaso >= PERIODO) {    // ya pasaron PERIODO ms?
    tPaso = ahora;     // anotar la hora
    posicion = (posicion + 1) % N_LEDS;     // avanzar al siguiente LED
  }

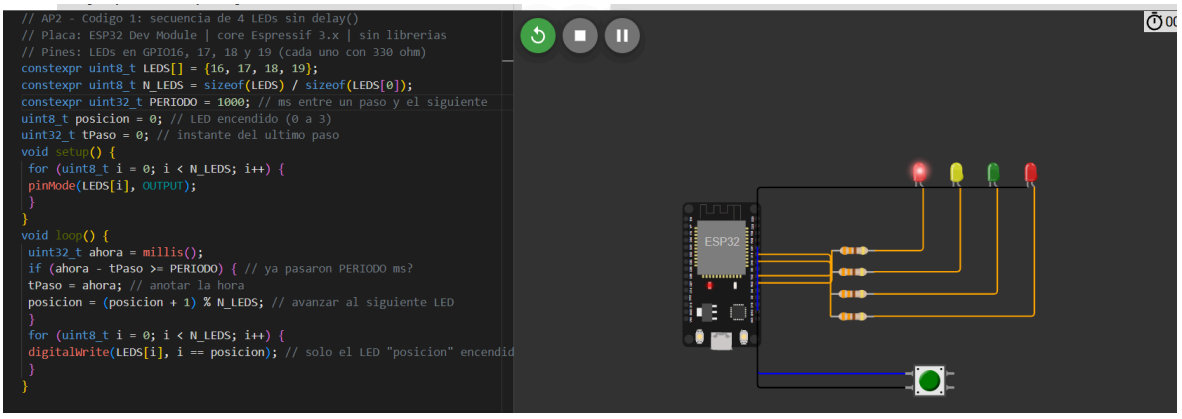
  for (uint8_t i = 0; i < N_LEDS; i++) {
    digitalWrite(LEDS[i], i == posicion); // solo el LED "posicion"
    encendido
```

```
    }
}
```

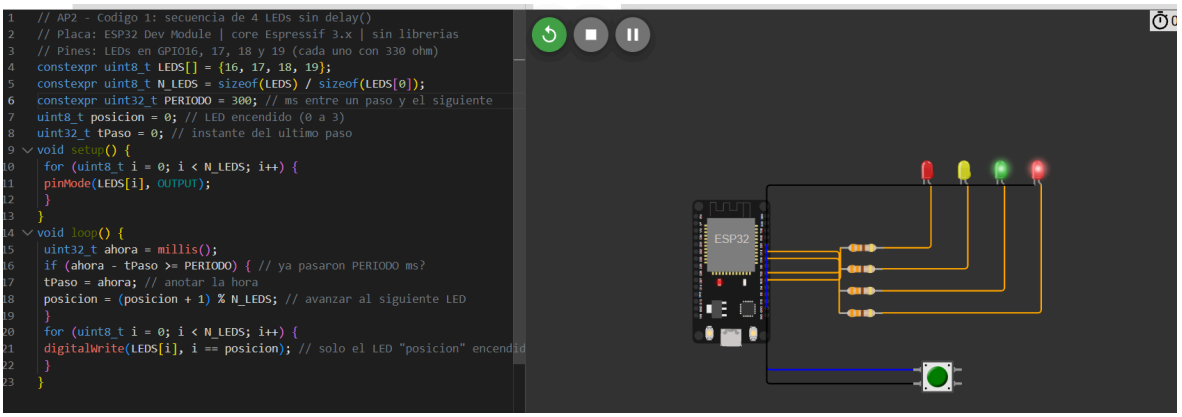
Tabla A. Efecto del periodo

PERIODO (ms)	¿Qué observan?
1000	Podemos observar que el tiempo en el que se cambian de led en led es aproximadamente 1 segundo es el más lento en cambiar
300	Aqui podemos observar que comienza desde el led amarillo y su tiempo de cambios es más rapido que el de 1000
100	Aquí observamos que es el más rápido para cambiar entre led a led

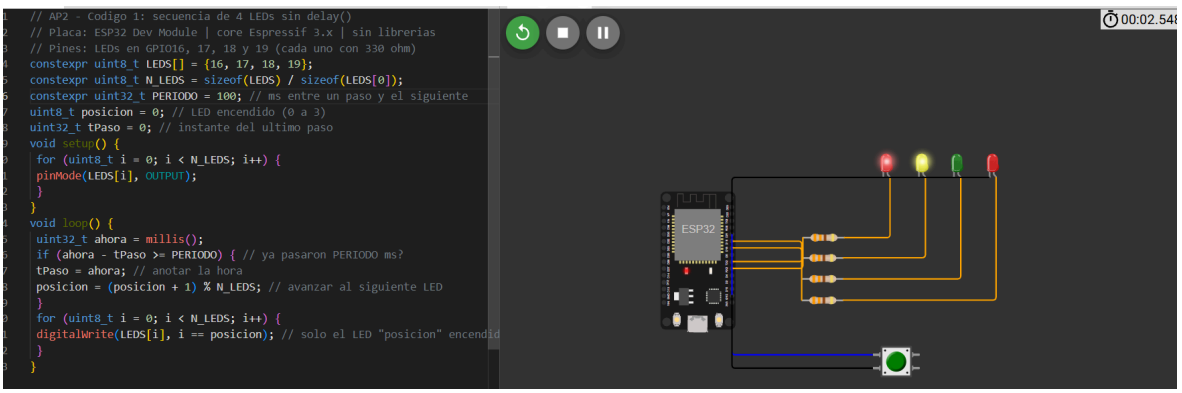
1000



300



100



Parte B. El pulsador cambia la velocidad (Wokwi)

En esta parte cada pulsación cambia la velocidad: lento (400 ms) → medio (150 ms) → rápido (60 ms) → lento...

B1. En el mismo proyecto, reemplacen el contenido de sketch.ino por el Código 2 y presionen ▸.

B2. Hagan clic sobre el pulsador varias veces y miren el Monitor Serie.

Resultado esperado: cada clic cambia la velocidad y el Monitor Serie imprime
Periodo: ... ms una sola vez por clic.

B3. Mantenga presionado el pulsador durante 3 s (clic sostenido).

Resultado esperado: la velocidad cambia una sola vez, aunque el botón siga presionado.

B4. Cambien T_REBOTE a 0, ejecuten, hagan 10 clics y cuenten cuántas líneas imprime el Monitor Serie. Anoten el resultado en la Tabla B.

B5. Vuelvan a dejar T_REBOTE = 30, repitan los 10 clics y completen la Tabla B.

B6. Tomen una captura del Monitor Serie con T_REBOTE = 30.

```
// AP2 - Codigo 2: juego de luces interactivo
// El pulsador cambia la velocidad: lento -> medio -> rapido -> lento...
// Placa: ESP32 Dev Module | core Espressif 3.x | sin librerias
// Pines: LEDs en GPIO16, 17, 18 y 19 (330 ohm); pulsador entre GPIO21 y
GND constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS     = sizeof(LEDS) /
sizeof(LEDS[0]); constexpr uint8_t PIN_BTN  = 21;
constexpr uint32_t T_REBOTE  = 30;                 // ms de antirrebote
constexpr uint32_t PERIODOS[] = {400, 150,         // ms: lento, medio,
60};                                               rapido
constexpr uint8_t N_VEL      = sizeof(PERIODOS) /
sizeof(PERIODOS[0]);

                             // LED encendido (0 a 3)
uint8_t posicion = 0;
uint8_t velocidad = 0;       // indice dentro de
uint32_t tPaso = 0;          PERIODOS
                             // instante del ultimo paso

bool     lecturaAnterior = HIGH;      // ultima lectura cruda del
bool     pulsador        =
uint32_t estadoEstable  /HIGH;       estado filtrado (HIGH = suelto)
tUltimoCambio            = 0;

// Devuelve true solo en el instante en que se presiona el
pulsador bool botonPresionado() {
  bool lectura = digitalRead(PIN_BTN);
  if (lectura != lecturaAnterior) {  // la senal se movio
    tUltimoCambio     = millis();
    lecturaAnterior = lectura;
```

Código 2. Placa: ESP32 Dev Module · core Espressif 3.x · sin librerías

```
  }
  if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable) {
    estadoEstable = lectura;// lleva 30 ms quieta: cambio real
    return estadoEstable == LOW;    // flanco de bajada = se presiono
  }
  return false;
}

void setup() {
  Serial.begin(115200);
  for (uint8_t i = 0; i < N_LEDS; i++) {
    pinMode(LEDS[i], OUTPUT);
  }
  pinMode(PIN_BTN, INPUT_PULLUP);   // suelto = HIGH, presionado = LOW
  Serial.println("AP2 lista. Presiona el pulsador.");
}

void loop() {
  // 1) Entradas
  if (botonPresionado()) {
    velocidad = (velocidad + 1) % N_VEL;
    Serial.print("Periodo: ");
    Serial.print(PERIODOS[velocidad]);
    Serial.println(" ms");
  }

  // 2) Logica temporizada
  uint32_t ahora = millis();
  if (ahora - tPaso >= PERIODOS[velocidad]) {
    tPaso = ahora;
    posicion = (posicion + 1) % N_LEDS;
  }

  // 3) Salidas
  for (uint8_t i = 0; i < N_LEDS; i++) {
    digitalWrite(LEDS[i], i == posicion);
  }
}
```

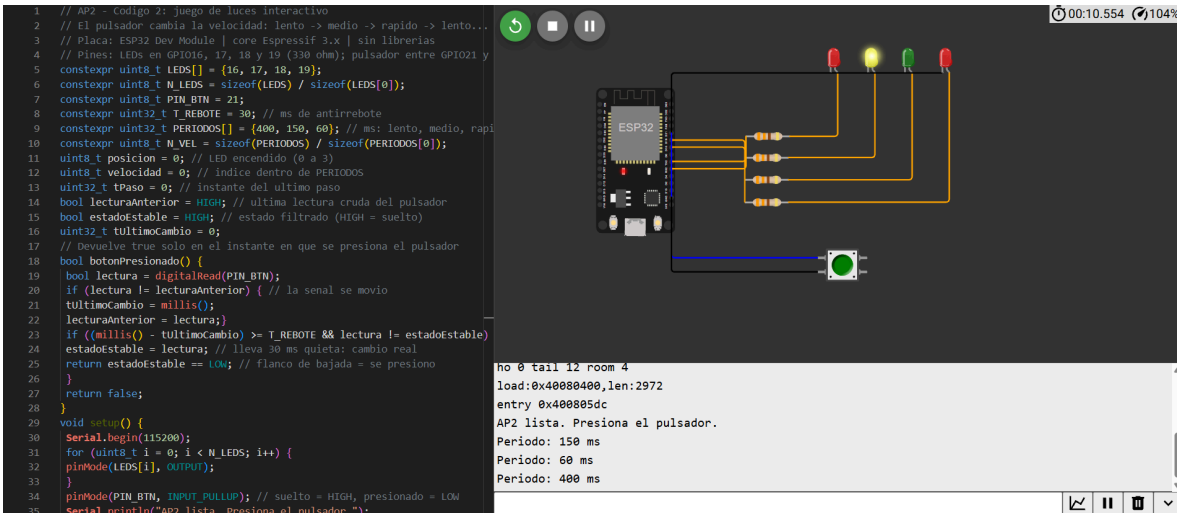
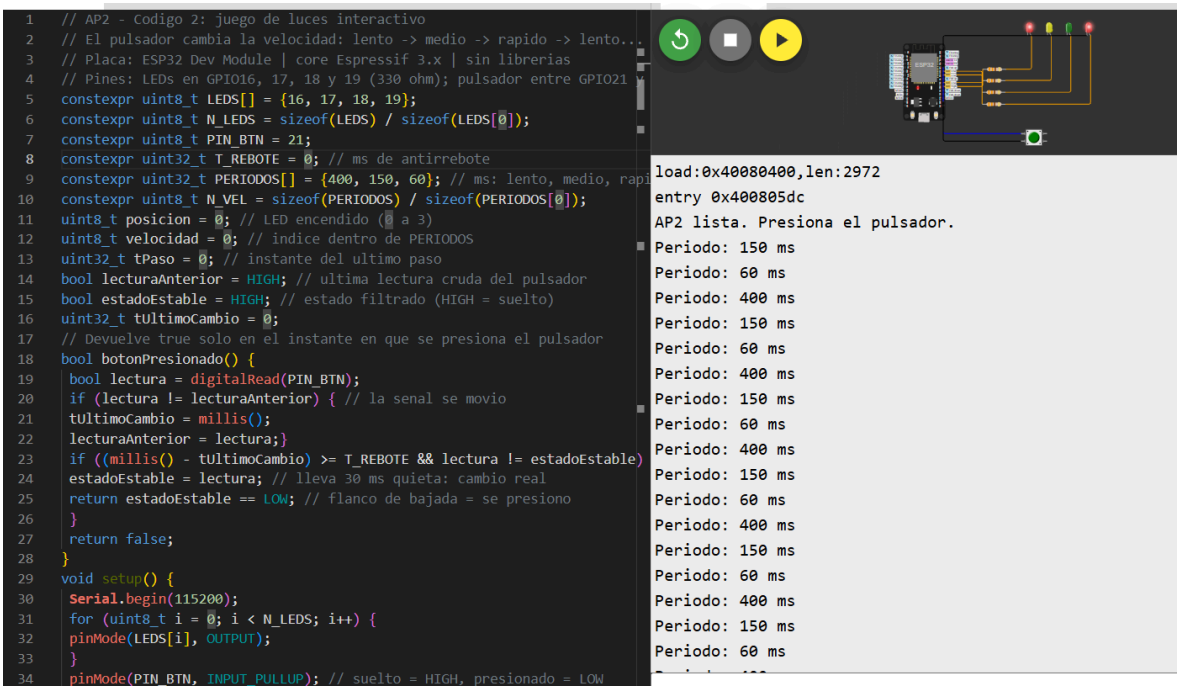


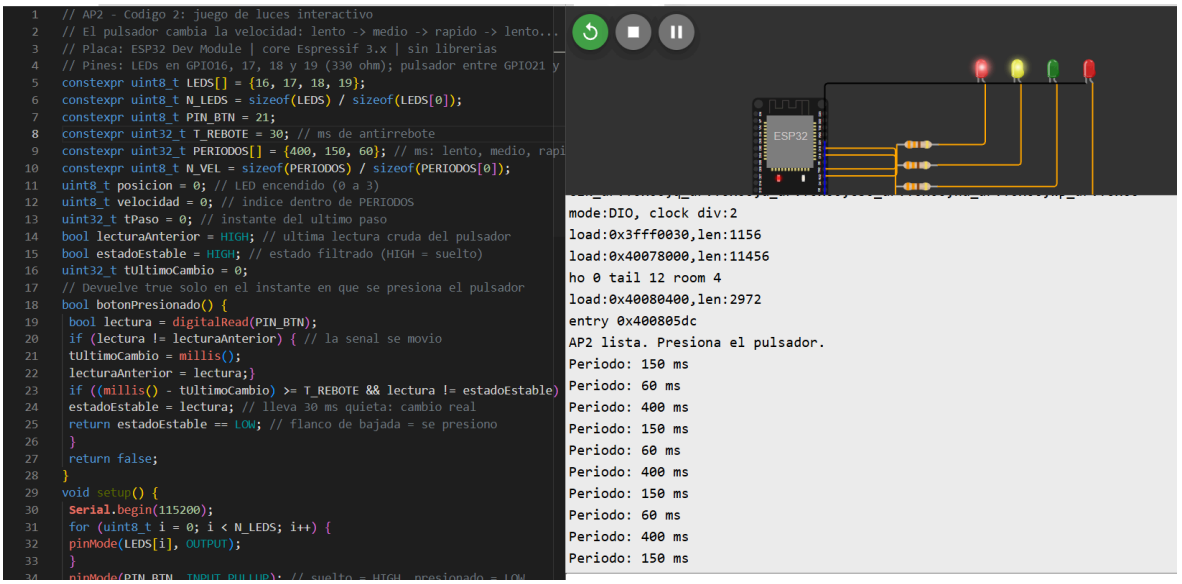
Tabla B. Efecto del antirrebote (10 clics)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	3 Lineas imprime en Consola	Si, Coinciden entre sí
30	1 Línea imprime en consola	Va cambiando el valor del periodo

T_REBOTE 0



T_REBOTE 30



Parte C. Prueba en la placa ESP32 real

Se carga en la placa el mismo Código 2, sin cambiar nada. La hace cada grupo que ya tenga su ESP32; los grupos sin placa entregan la evidencia de Wokwi y montan el circuito en la siguiente sesión presencial.

C1. Con el USB desconectado, armen el circuito en la protoboard según la Tabla de conexiones. Revisen la polaridad: pata larga del LED hacia la resistencia y el GPIO, pata corta hacia GND.
C2. Conecten el cable USB. En Arduino IDE elijan Herramientas → Placa → ESP32 Dev Module y el puerto que aparezca (COMx en Windows, /dev/ttyUSB0 en Linux).

C3. Abran el Código 2 y presionen Subir. Si aparece Connecting y no avanza, mantengan presionado el botón BOOT de la placa hasta que empiece a cargar.

C4. Abran el Monitor Serie a 115200 baudios y repitan los pasos B2 y B3.

Resultado esperado: el mismo comportamiento que en Wokwi.

C5. Graben un video corto (30 s como máximo) del montaje funcionando.

Si la carga falla, revisen la diapositiva «Cuando el programa no sube» de la sesión 1: cable solo de carga, driver CP210x o CH340, puerto ocupado por otro Monitor Serie.

Parte D. Entrega

D1. Creen un repositorio en GitHub por pareja, llamado AP2-SistemasDigitales-<apellido1>-<apellido2>, con esta estructura:

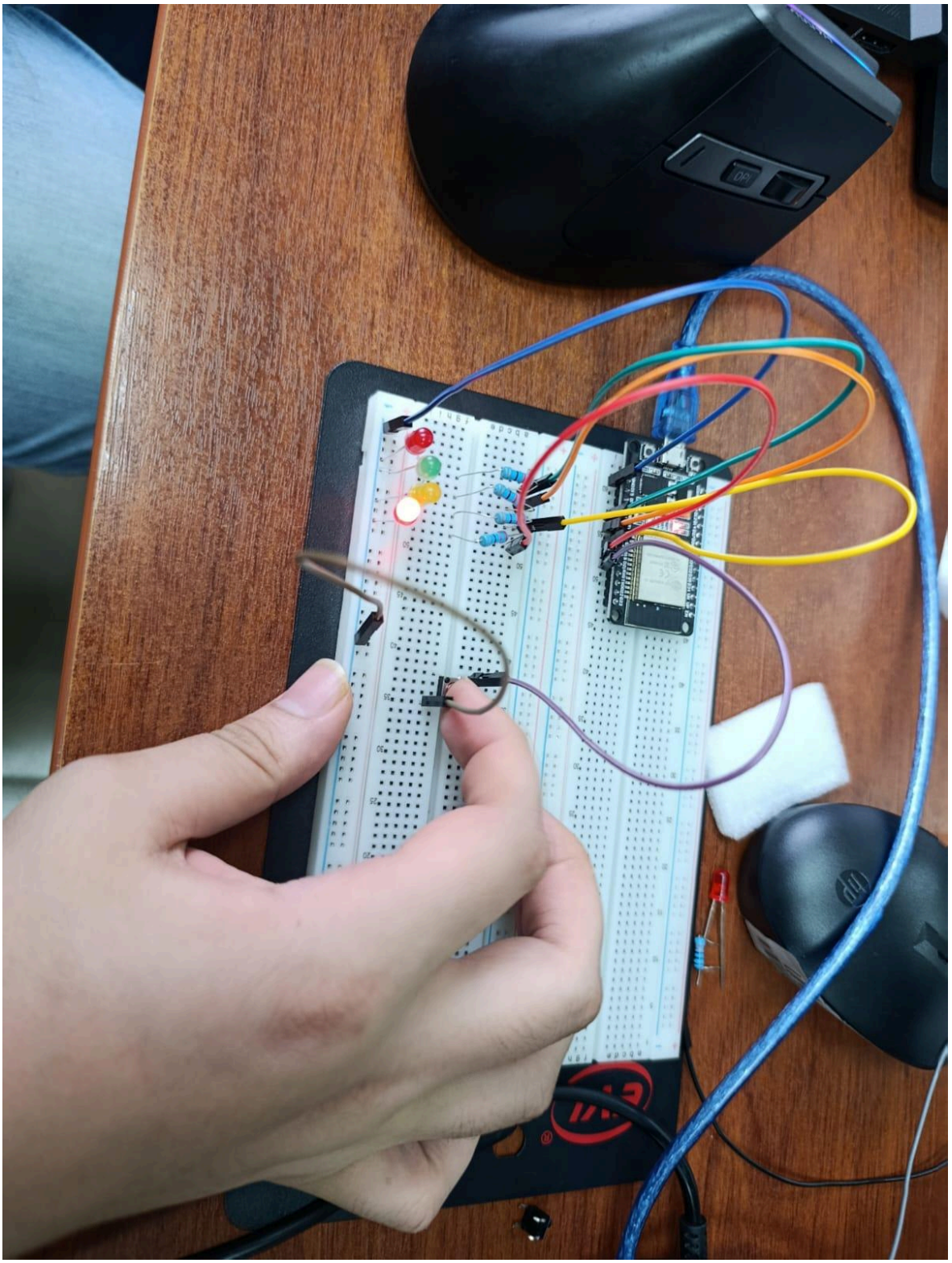
- codigo1/AP2_Codigo1.ino
- codigo2/AP2_Codigo2.ino
- wokwi/diagram.json
- evidencias/ (capturas de Wokwi y, si hubo Parte C, el video o su enlace)
- informe/AP2_Informe.pdf

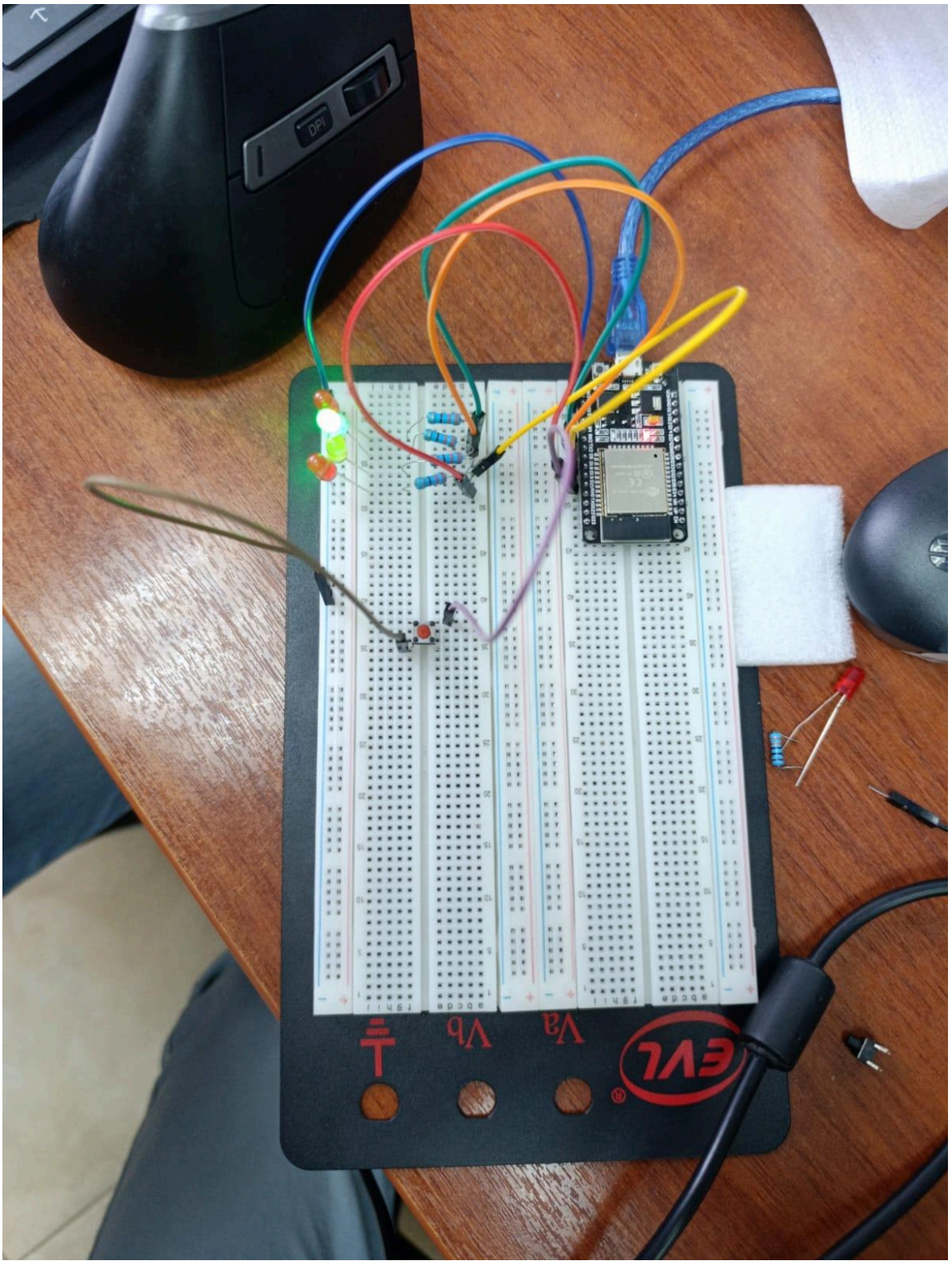
D2. Suban al EVA, en la tarea de la AP2, el enlace del repositorio y el informe en PDF.

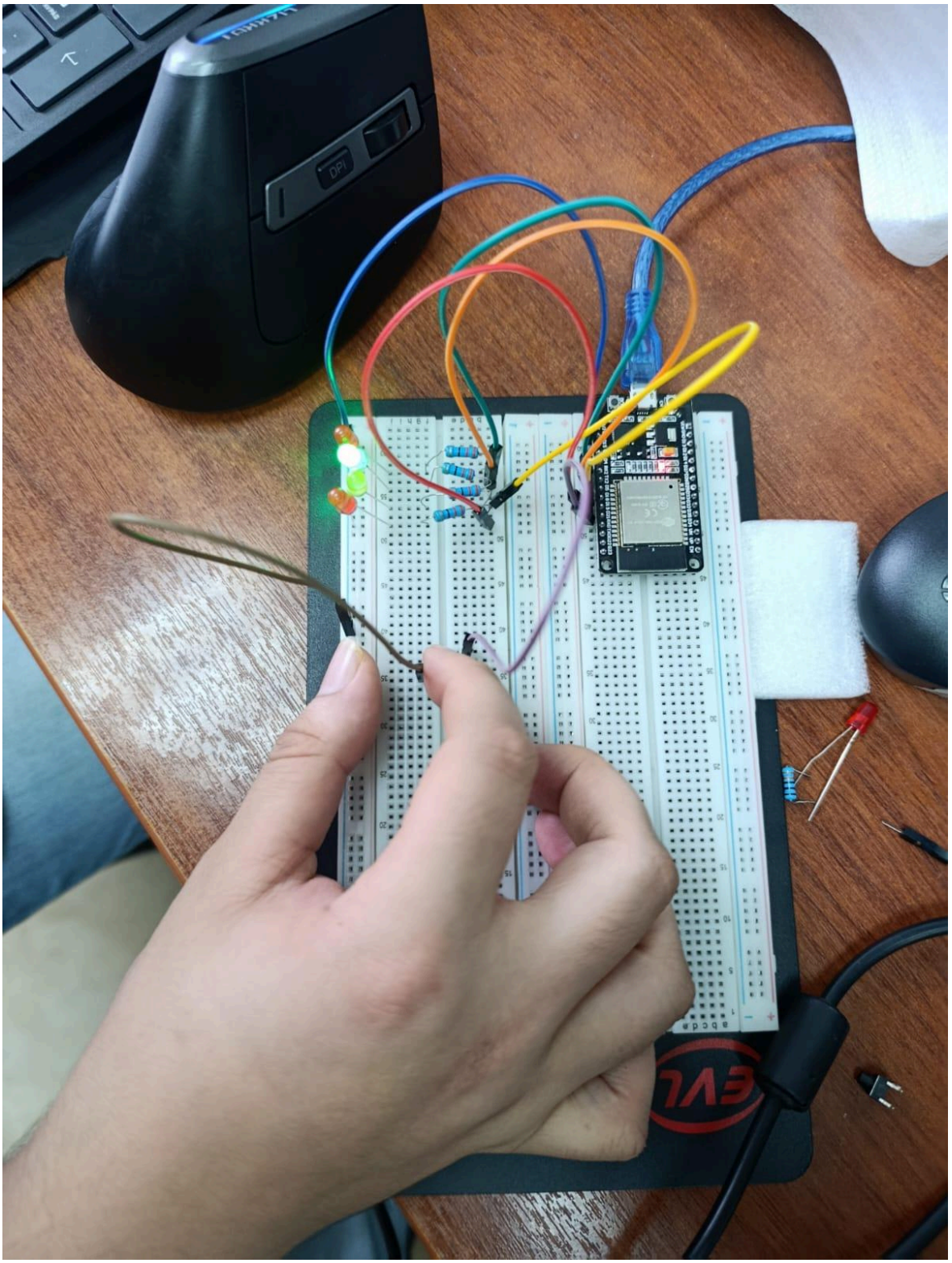
7. Resultados esperados:

- Código 1: un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16… cada 0,3 s; con PERIODO = 1000 y 100 la secuencia se vuelve más lenta o más rápida (Tabla A).
- Código 2: cada clic cambia la velocidad (lento 400 ms → medio 150 ms → rápido 60 ms → lento…) y el Monitor Serie imprime Periodo: ms una sola vez por clic.
- Clic sostenido: la velocidad cambia una sola vez, aunque el botón siga presionado.
- Placa real (Parte C): el mismo comportamiento que en Wokwi.

Se debe entregar el esquema de conexiones, los códigos, las evidencias (capturas de Wokwi y, si hubo Parte C, el video) y las Tablas A y B completas.}




DPI
Vb
Va



- **Uso de IA**

  Declaración de uso de la IA

  Declaro que, para la elaboración de esta tarea, hice uso de una herramienta de Inteligencia

  Artificial (Gemini y Grok) únicamente como apoyo para:

  - Busca de documentos de interés
  - Mejorar la redacción
  - Síntesis de ideas.
  - Guia para realizar la práctica

  El uso de la IA se realizó respetando el uso ético y responsable de la Inteligencia Artificial.

- **Link Del Github**

LINK

https://github.com/Mzero11/AP2-SistemasDigitales--Chingo-Guachizaca-Valarezo-Villamagua-Silva.git

8. Preguntas de Control:

1. ¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?

El Código 1 usa millis() para implementar una temporización no bloqueante. Esto permite que el ciclo loop() se ejecute continuamente sin congelar la CPU. Si se utilizara delay(400), el programa se detendría por completo durante 400 ms en cada LED; por lo tanto, si el pulsador fuera presionado durante esa pausa, la lectura no se registraría y la respuesta del sistema sería lenta o insensible.

2. El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?

- Pulsador suelto: Lee HIGH (1 / 3,3 V). Esto se debe a que la instrucción pinMode(PIN_BTN, INPUT_PULLUP) activa la resistencia pull-up interna del ESP32, manteniendo la línea conectada internamente a un nivel alto.
- Pulsador presionado: Lee LOW (0 / 0 V). Al presionar el botón se cierra el circuito y se conecta directamente el GPIO21 a la línea de tierra (GND).

3. ¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase.

Los pines GPIO 34 al 39 del ESP32 son exclusivamente de entrada (*input-only*) y no poseen resistencias de pull-up o pull-down internas. Si se mantiene el código sin cambios, la instrucción INPUT_PULLUP no activará ninguna resistencia interna en el GPIO34, dejando el pin en estado flotante (*floating*) y provocando lecturas aleatorias e inestables cuando el pulsador esté suelto.

4. En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?

Lo garantiza la lógica de detección de flanco dentro de la función botonPresionado(). En particular, la condición: if ((millis() - tUltimoCambio) > T_REBOTE && lectura != estadoEstable) junto con la instrucción estadoEstable = lectura; y el retorno return estadoEstable == LOW;. Cuando el botón se mantiene presionado, la señal permanece constante (lectura == estadoEstable), provocando que la condición no se cumpla y la función retorne false en las lecturas siguientes.

5. Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?

Al establecer $T_REBOTE = 0$, se desactiva el filtro por software de antirrebote. Debido a que los contactos mecánicos del pulsador rebotan físicamente durante unos milisegundos al ser presionados, la ausencia de un tiempo de estabilización causa que el microcontrolador interprete cada pequeña oscilación como una nueva pulsación, imprimiendo en el Monitor Serie más líneas que los clics reales efectuados.

6. **Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?**

Cálculo de corriente: La tensión sobre la resistencia (V_R) equivale a la diferencia entre el voltaje de alimentación y el voltaje directo del LED:

$$V_R = V_{CC} - V_f = 3,3V - 2,0V = 1,3V$$

Aplicando la ley de Ohm ($I = \frac{V_R}{R}$):

$$I = \frac{1,3V}{330\Omega} \approx 0,003939A \approx 3,94mA$$

¿Por qué no conviene un LED azul?: Los LEDs azules presentan un voltaje directo (V_f) alto, típicamente de 3,0 V a 3,4 V. Al operar a 3,3 V, la diferencia de voltaje en la resistencia ($V_{CC} - V_f$) resulta casi nula o insuficiente para permitir el flujo de corriente necesario, haciendo que el LED encienda muy tenue o casi no encienda

7. **Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.**

Wokwi opera en un entorno simulado ideal, libre de ruido e imperfecciones electromagnéticas. En la placa real existen factores mecánicos y eléctricos reales, tales como falsos contactos en la protoboard o cables jumpers, tolerancias en las resistencias/LEDs, rebotes mecánicos más impredecibles, y el requerimiento de controladores USB (CP210x o CH340) o presionar el botón `BOOT` para realizar la carga del programa.

8. **El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.**

Se puede reducir la frecuencia de reloj de la CPU de 240 MHz a 80 MHz o menor, desactivar los periféricos inalámbricos no utilizados mediante `WiFi.mode(WIFI_OFF)` y `btStop()`, o poner la CPU en modo de bajo consumo (*Light Sleep*) entre cambios de LED, despertándola únicamente cuando ocurra un evento de temporización o mediante una interrupción externa en el GPIO21 provocada por el pulsador. n

9. Entregables y estructura del informe

El informe debe tener la siguiente estructura:

- Portada con los datos de la UNL y los nombres de la pareja.
- Objetivo.
- Materiales y software usados.
- Esquema de conexiones: captura de Wokwi y Tabla de conexiones.
- Desarrollo: Partes A, B y C, con las Tablas A y B completas y las capturas.
- Respuestas a las preguntas de control.
- Conclusiones (mínimo tres).
- Declaración de uso de IA generativa: qué herramienta usaron, para qué y en qué partes; si no usaron ninguna, decírlo.
- Referencias.

Entrega: el repositorio de GitHub con la estructura de la Parte D y, en la tarea de la AP2 del EVA, el enlace del repositorio y el informe en PDF.

10. Evaluación

Calificación sobre 10 puntos. Cada nivel vale el porcentaje indicado de los puntos del criterio. Los grupos que todavía no tienen placa no pierden puntos por la Parte C: presentan el video en la siguiente sesión.

Criterio	Puntos	Excelente (100 %)	Satisfactorio (70 %)	En desarrollo (40 %)	No presenta (0 %)
Parte A: circuito y Código 1	1,5	La secuencia funciona y la Tabla A está completa y bien explicada	Funciona; la Tabla A está incompleta	Funciona a medias o sin Tabla A	No presenta

Parte B: Código 2 y pulsador	2,0	Una pulsación = un cambio; Tabla B completa y coherente	Funciona; Tabla B incompleta	El pulsador falla o no hay Tabla B	No presenta
Evidencias	1,5	Capturas de Wokwi claras y video de la Parte C (si tienen placa)	Faltan algunas capturas	Evidencias poco claras	No presenta
Preguntas de control	3,0	Las 8 respuestas son correctas y justificadas	6 o 7 correctas	3 a 5 correctas	Menos de 3
Conclusiones y declaración de IA	1,0	Tres conclusiones propias y declaración completa	Conclusiones genéricas o declaración incompleta	Una sola conclusión o sin declaración	No presenta
Entrega	1,0	Repositorio con la estructura pedida y PDF en el EVA	Falta un elemento	Faltan varios elementos	No presenta

11. Bibliografía

1. Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.
2. Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.
3. Espressif Systems. (s. f.). ESP32 Series Datasheet. https://www.espressif.com/en/support/documents/technical-documents
4. Espressif Systems. (s. f.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/
5. CodeMagic LTD. (2025). Wokwi – documentación. https://docs.wokwi.com
6. Arduino. (2024). Arduino Language Reference. https://www.arduino.cc/reference/

12. Elaboración y Aprobación

Elaborado por	Ing. Luis Darío Sinche Cueva Docente	Validar únicamente en Firmalec Firmado electrónicamente por: LUIS DARIO SINCHE CUEVA
Revisado por	Ing. Ramiro Rene Rivera Guamán Técnico Docente	Validar únicamente en Firmalec Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Ing. Edison Leonardo Coronel Romero Director de Carrera	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con Firmalec

Anexo A. diagram.json de Wokwi

Peguen este contenido en la pestaña diagram.json (paso A2).

```
{
  "version": 1,
  "author": "Sistemas Digitales UNL - AP2",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {}
},
    { "type": "wokwi-resistor", "id": "r1", "top": 60, "left": 200, "attrs": {
"value": "330" } },
    { "type": "wokwi-resistor", "id": "r2", "top": 90, "left": 200, "attrs": {
"value": "330" } },
    { "type": "wokwi-resistor", "id": "r3", "top": 120, "left": 200, "attrs": {
"value": "330" } },
    { "type": "wokwi-resistor", "id": "r4", "top": 150, "left": 200, "attrs": {
"value": "330" } },
    { "type": "wokwi-led", "id": "led1", "top": -60, "left": 300, "attrs": {
"color": "red" } },
    { "type": "wokwi-led", "id": "led2", "top": -60, "left": 350, "attrs": {
"color": "yellow" } },
    { "type": "wokwi-led", "id": "led3", "top": -60, "left": 400, "attrs": {
"color": "green" } },
    { "type": "wokwi-led", "id": "led4", "top": -60, "left": 450, "attrs": {
"color": "red" } },
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 220, "left": 300, "attrs": {
"color": "green" } }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:RX2", "r1:1", "orange", [] ],
    [ "esp:TX2", "r2:1", "orange", [] ],
    [ "esp:D18", "r3:1", "orange", [] ],
    [ "esp:D19", "r4:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "r2:2", "led2:A", "orange", [] ],
    [ "r3:2", "led3:A", "orange", [] ],
    [ "r4:2", "led4:A", "orange", [] ],
    [ "led1:C", "esp:GND.1", "black", [] ],
    [ "led2:C", "esp:GND.1", "black", [] ],
    [ "led3:C", "esp:GND.1", "black", [] ],
    [ "led4:C", "esp:GND.1", "black", [] ],
    [ "btn1:1.l", "esp:D21", "blue", [] ],
    [ "btn1:2.l", "esp:GND.1", "black", [] ]
  ],
  "dependencies": {}
}
```